

LangChain & Agentic AI – From Basics to Advanced

demo- 9 Notes

=====

Most Important Points to Remember

Point 1: A prompt is the input/instruction given to an AI model.

Point 2: A static prompt is fixed.

Point 3: A dynamic prompt contains values that can change.

Point 4: {topic} is an example of an input variable/placeholder.

Point 5: PromptTemplate helps us create reusable dynamic text prompts.

Point 6: PromptTemplate does not generate the AI answer. It will generate final prompt.

For this we have to use `pt.invoke()`

Point 7: The generated prompt is passed to the model. For this we have to use `llm.invoke()`

Point 8: For a very simple requirement, a normal Python f-string may be enough. Use LangChain abstractions when they help the application structure.

Template + Data = Final Prompt

Final Prompt + LLM = AI Response

Multiple variables:

A prompt template can contain more than one placeholder.
Each placeholder can receive a different value at runtime.

eg:

Explain {Topic}

Explain Python in Telugu

Explain {topic} in {language}

Explain Python in Telugu for Beginner level in Simple Style

Explain {topic} in {language} for {level} level in {style} Style

Analogy:

Invitation Template

Hello {name},

You are Invited to {event}

on {date}

at {place}.

=====

import os

from langchain_openai import ChatOpenAI

from langchain_core.prompts import PromptTemplate

```
llm = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
prompt_template = PromptTemplate.from_template("Explain  
{topic} in {language} in {style} style")
```

```
#print(prompt_template.input_variables) #['language', 'style',  
'topic']
```

```
prompt = prompt_template.invoke(  
    {  
        "topic": "Python",  
        "style": "simple",  
        "language": "Telugu"  
    }  
)
```

```
print(prompt.to_string())
```

```
ai_message = llm.invoke(prompt)
print(ai_message.content)
```

```
=====
```

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
llm = ChatOpenAI(
```

```
    model="gpt-4o-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
prompt_template = PromptTemplate.from_template("Explain  
{topic} in {language} in {style} style")
```

```
#print(prompt_template.input_variables) #['language', 'style',  
'topic']
```

```
topic = input("Enter Topic:")
```

```
language = input("Enter Language:")
```

```
style = input("Enter Style:")
```

```
prompt = prompt_template.invoke(  
{
```

```
{
```

```
    "topic":topic,
```

```
    "style":style,
```

```
    "language":language
```

```
}
```

```
)
```

```
print(prompt.to_string())
```

```
ai_message = llm.invoke(prompt)
```

```
print(ai_message.content)
```

```
=====
```

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
# Create model
```

```
llm = ChatOpenAI(
```

```
    model="gpt-4o-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
# Create reusable template
```

```
template = PromptTemplate.from_template(
```

```
    """
```

```
    Teach me {topic}.
```

```
    Student Level: {level}
```

```
    Language: {language}
```

Explanation Style: {style}

Requirements:

1. Explain in easy language.
2. Give 5 important points.
3. Give 2 real-life examples.
4. Mention one common mistake.
5. End with a one-line conclusion.

""""

)

Take user inputs

topic = input("Enter topic: ")

level = input("Enter your level: ")

language = input("Enter language: ")

style = input("Enter explanation style: ")

Generate prompt

```
prompt = template.invoke(  
    {  
        "topic": topic,  
        "level": level,  
        "language": language,  
        "style": style  
    }  
)
```

```
# Inspect generated prompt
```

```
print("\n===== GENERATED PROMPT =====")  
print(prompt.to_string())
```

```
# Send to model
```

```
response = llm.invoke(prompt)
```

```
# Display response
```

```
print("\n===== AI RESPONSE =====")
```

```
print(response.content)
```

```
=====
```

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
llm = ChatOpenAI(
```

```
    model="gpt-4o-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
template = PromptTemplate.from_template(
```

```
    """
```

```
    Create a simple travel plan for {place}.
```

```
    Duration: {days} days
```

```
    Budget: {budget}
```

Include:

- Places to visit
- Suggested daily plan
- Food suggestions
- Simple travel tips

""""

)

```
place = input("Enter destination: ")
```

```
days = input("Enter number of days: ")
```

```
budget = input("Enter budget type (low/medium/high): ")
```

```
prompt = template.invoke(
```

```
{
```

```
    "place": place,
```

```
    "days": days,
```

```
    "budget": budget
```

```
    }  
)  
  
response = llm.invoke(prompt)  
  
print("\nTravel Plan:")  
print(response.content)  
  
=====
```

#Interview Helper Program

```
import os  
  
from langchain_openai import ChatOpenAI  
from langchain_core.prompts import PromptTemplate  
  
llm = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
template = PromptTemplate.from_template(
    """
    Generate {count} interview questions on {technology}
    for a {level} candidate.

    For every question:
    - Give the question
    - Give a simple answer
    """
)
```

```
technology = input("Enter technology: ")
```

```
level = input("Enter level: ")
```

```
count = input("How many questions?: ")
```

```
prompt = template.invoke(
    {
```

```
        "technology": technology,  
        "level": level,  
        "count": count  
    }  
)
```

```
response = llm.invoke(prompt)  
print("\n AI Response")  
print(response.content)
```

```
=====
```

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
llm = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")
```

)

```
template = PromptTemplate.from_template(
```

```
    """
```

```
    Suggest 10 restaurant names.
```

```
    Cuisine: {cuisine}
```

```
    Style: {style}
```

```
    Location: {location}
```

```
    Names should be:
```

- Easy to remember
- Attractive
- Suitable for the restaurant

```
    """
```

```
)
```

```
cuisine = input("Enter cuisine: ")
```

```
style = input("Enter restaurant style: ")
```

```
location = input("Enter location: ")
```

```
prompt = template.invoke(
```

```
{
```

```
    "cuisine": cuisine,
```

```
    "style": style,
```

```
    "location": location
```

```
}
```

```
)
```

```
response = llm.invoke(prompt)
```

```
print("\nRestaurant Name Ideas:")
```

```
print(response.content)
```